Measuring the Carbon Footprint of Programming Languages

Anuj Khadka
Computer Science and Business Analytics
Caldwell University
akhadka@caldwell.edu

Vladislav D. Veksler
Chair of Department of CSIS
Caldwell University
vveksler@caldwell.edu

Darryl Aucoin
Chair of Department of Natural Science
Caldwell University
daucoin@caldwell.edu

Abstract

This project investigates how programming language choice affects the carbon footprint produced during a program execution. Although software is often seen as intangible, every program consumes energy, ultimately contributing to carbon emissions. As the digital system continues to grow, understanding the environmental impact of software has become increasingly important. This experiment studies how identical algorithms behave when implemented in different programming languages running on the same hardware. By measuring and estimating the carbon emissions, the project tends to explore whether programming language designs play an important role in software sustainability, or if it is mostly driven by heavy algorithms. This research combines the aspects of computer science and environmental science to better understand the hidden environmental cost of everyday computational tasks and encourage more sustainable and responsible software development practices.

Keywords: carbon footprint, software sustainability, energy consumption, carbon intensity, programming languages, algorithms

1 Introduction

As the tech continues to evolve, software becomes an increaingly central part of our lives. From the apps on our phones to the software running our cars, it's everywhere. Everytime a piece of code runs, it triggers energy usage inside the CPU, GPU, and other hardware components [1]. This subsequent usage of software adds up to the carbon emission a device generates [2]. The carbon footprint of software is a growing concern, as it contributes to climate change and has a significant impact on our planet.

The scale of this concern is larger than most people realize. The information and communication technology (ICT) sector accounts for an estimated of 2.1% to 3.9% of the global greenhouse gas emissions [3]. The global data center energy consumption alone is projected to reach around 945TWh by 2030, which is a significant increase from 153TWh in 2005, 205TWh in 2018. [4, 5]. Unlike the visible emissions from vehicles, software carbon is invisible. There is no smoke when a program runs, yet every computation contributes indirectly to the same atmosphere. This invisibility is precisely what makes it dangerous. It does not trigger the same urgency that the physical pollution does, even as the number grows.

One of the most consequential and least-studied of those decisions is the choice of programming language. The carbon cost of a running program is not determined by a hardwar alone. It is also shaped by how efficient the language runtime executes instructions, manages memory, and utilizes the CPU.In a landmark empirical study, Pereira et al. [6] benchmarked 27 programming languages across 10 problems drawn from the Computer Language Benchmarks Game [7], measuring energy consumption via Intel's Running Average Power Limit (RAPL) interface. They found that the choice of programming language can have a significant impact on energy consumption, with some languages consuming up to 2.5 times more energy than others for the same task. For example, they found that C and Rust were among the most energy-efficient languages, while Python and JavaScript were among the least efficient. Python consumed approximately 75.88 times more energy per benchmark than C while JavaScript consumed approximately 4.45 times more energy per benchmark than C [6]. Since carbon emissions are directly proportional to energy consumption, these differences are not just technical concern, but also a carbon concern. A Python program consuming about 75 times more energy than an equivalent C program does not just run slower, it emits 75 times more carbon for every execution. Now, multiply that by the millions of times a piece of software runs, and the carbon cost becomes staggering.

Prior work in this area, including Pereira et al., reports energy consumption in joules or kilowatt-hours but none of it completes the translation into carbon footprint. This

is a critical gap. Energy consumption is the underlying driver, but carbon footprint is the ultimate impact. This unit is most directly relevent to the clomate policy and environmental decision making. Henderson et al. showed that the emissions can be reduced by up to 30 times just by running the experiments in the locations powered by more renewable energy sources [8].

This paper presents a controlled empirical study comparing the energy consumption and the carbon footprint of six programming languages (C, Rust, Go, Java, Javascript, and Python) across six different algorithms and three imput sizes. The energy consumption is measured using Intel's RAPL (accessed through LibreHardwareMonitor [9]), and real-time carbon intensity data is fetched from the Electricity Maps API [10] for the New York grid zone (US-NY-NYIS). Each of the 108 experimental cells (6 languages $\times$ 6 algorithms $\times$ 3 input sizes) was replicated 115 times, yielding 12,420 total measurements. Statistical analysis applies Welch's t-tests with Bonferroni correction across all 15 pairwise language comparisons. The results show that 14 of 15 language pairs are statistically distinguishable in carbon emissions. The sole exception is C versus Rust ($t = -0.313$, $p = 0.754$), confirming that the two languages are carbon-equivalent. Python is the most carbon-intensive language tested, producing on average 213$\times$ more carbon emissions than C for identical workloads.

2 Background

In the following sections, we describe the energy consumption model, carbon intensity assumptions, programming languages, and the implementations of the six algorithms used in this study.

2.1 Energy Consumption in Software

To quantify the energy consumed by a software program, we adopt the standard defination of energy as the capacity to do work.

$$E_{\text{energy}} = P_{\text{power}} \times t_{\text{time}} \tag{1}$$

where E is energy measured in Joules(J), Power is rate of energy consumption measured in Watts(W), and Time is the duration of execution measured in seconds(s)[11]. In the context of software execution, the T refers to the wall time, the elapsed time from the first instruction of the measured code path to the last, including CPU computation, memory access, and all other processor activity during execution [12].

The energy consumed by a CPU package during the program execution can therefore be decomposed as the integral of instantaneos power draw over the execution window. Since power draw is not constant across languages or algorithms, a key finding of Pereira et al. [6], energy efficiency cannot be inferred from execution time alone. A faster program is not necessarily a more energy efficient one. This encourages us to direct hardware level energy measurement via RAPL.

All experiments were conducted on a desktop with the following specifications: Windows 11 Enterprise 25H2 (Build 26200.8246) operating system, with 32.0 GB RAM, and an Intel Core `i7-14700K` CPU @ 3.40 GHz.

2.2 Carbon Intensity

To translate the energy measurements into carbon emissions, we apply the SCI framework.

The relationship between energy and carbon depends on how that energy is generated. Energy produced from hydroelectric or wind generation carries near-zero carbon intensity while the electricity from coal or natural gas has 800-1000 gCO2/kWh [13]. The carbon intensity of an electricity grid represents the average emissions intensity of all generation sources dispatched at a given moment, weighted by their contribution to total supply. This intensity is measured in grams of CO2 equivalent per kilowatt-hour, gCO2/kWh.

We obtain real-time carbon intensity from the Electricity Map API [10], which provides grid-zone-level carbon intensity updates every 30 minutes. Our experiment used zone US-NY-NYIS (New York Independent System Operator), which had a carbon intensity of `300-306` gCO2/kWh during our overnight experiment window (10pm - 6am). The gCO2 metric is computer from RAPL PKG energy readings as follows.

$$\text{gCO}_2\text{e} = \frac{E_{\text{J}}}{3{,}600{,}000} \times I_{\text{carbon intensity (gCO2/kWh)}} \tag{2}$$

where I is the grid carbon intensity in gCO2/kWh. This methodology is consistent with the Green Algorithms framework [14] and the Software Carbon Intensity (SCI) specification [15]

2.3 Programming Language

The programming languages chosen for this study are C, Python, Javascript, Java, Go, and Rust. These languages were selected to represent a range of programming paradigms, performance characteristics, and popularity in the software development community. Each language has its own unique features and trade-offs that may influence the energy consumption and carbon footprint of the algorithms implemented in them [16]. The performance characteristics of these languages are summarized in Table .

Table 1: Programming languages, their performance characteristics, and popularity.

Language	Paradigm	Runtime	Version
C	Manual Memory, Compiled	GCC	14.3.0
Rust	Ownership, Compiled	Rustup	1.94.0
Python	Interpreted	CPython	3.12.10
Javascript	Dynamic, V8	Node.js	24.14.0
Java	OOP, JVM	OpenJDK	21.0.10
Go	GC	Go	1.26.1

GCC was installed via WinLibs (MingGW-w64 distribu-